

Aula 1 — Fundamentos da Qualidade

"Por que sua suíte E2E é lenta e flaky?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Engenharia de Qualidade e Testes de Software · 24/09/2026

Bem-vindo(a) à disciplina!

Disciplina totalmente online **ao vivo**, 6 encontros síncronos de 3h30 (quintas, 19h-22h30) + trabalho assíncrono guiado entre aulas.

Estrutura:

- 6 aulas × 3h30 cada = 21h ao vivo + 3h assíncrono guiado = 24h totais
- Leituras dirigidas e labs entre um encontro e outro
- Office hours assíncrono via e-mail e fórum Canvas

| Bom momento pra começar: agora.

Agenda da aula (3h30)

- **90min** — Fundamentos: ISO/IEC 25010, Pirâmide/Trophy/Honeycomb, POM/Screenplay (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on em sala: setup Playwright (live coding com replicação pelos alunos)
- **15min** — Wrap-up + preview Aula 2

Objetivos da Aula 1

- **Compreender** ISO/IEC 25010 (8 atributos de qualidade)
- **Diferenciar** Pirâmide de Cohn, Testing Trophy (Dodds), Honeycomb (Spotify)
- **Distinguir** web testing vs mobile testing
- **Aplicar** Page Object Model e Screenplay Pattern com critério
- **Medir** flakiness e propor mitigação

Quiz inicial — você concorda?

| *"95% line coverage = teste bom."*

| *"Selenium é o padrão da indústria em 2026."*

| *"Sleep entre ações resolve flaky tests."*

| *"Pirâmide com 80% E2E é segura."*

| *"IA gera testes que sempre funcionam."*

Spoiler: todas falsas. Veremos por quê.

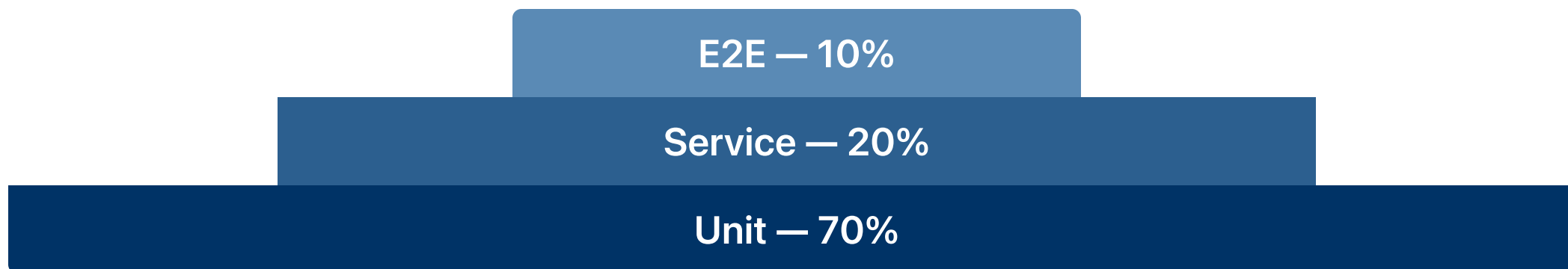
ISO/IEC 25010:2023 — qualidade de produto

8 atributos:

1. **Functional Suitability** — completude funcional
2. **Performance Efficiency** — eficiência de recursos
3. **Compatibility** — interoperabilidade
4. **Usability** — facilidade de uso
5. **Reliability** — disponibilidade, robustez
6. **Security** — proteção contra ameaças
7. **Maintainability** — facilidade de mudança
8. **Portability** — adaptabilidade entre ambientes

| Norma é vocabulário comum. Use em Test Strategy Documents.

Pirâmide de testes (Cohn) — clássica web

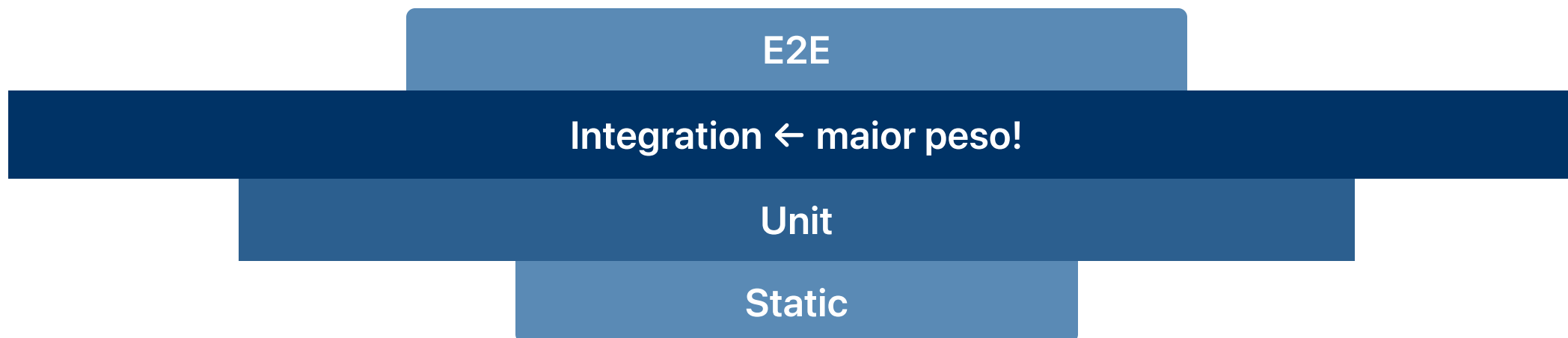


Por que pirâmide? E2E é caro, lento, frágil. Unit é barato, rápido, focado.

Aplica-se bem em projetos com bom design (interfaces estáveis, dependências claras).

Testing Trophy — Kent C. Dodds

Alternativa para apps JavaScript modernos:



Argumento: integration tests dão mais confidence per dollar em apps com muitos mocks de unit.

Não é "pirâmide errada" — é trade-off contextual.

Honeycomb — Spotify

Para microsserviços:

Integrated — poucos mas vitais

Integration ← grande peso

Implementation ← grande peso (= unit)

| Spotify defende que microsserviços precisam de balanço unit + integration; não de muitos E2E.

Quando usar qual?

Estratégia	Bom para
Pirâmide Cohn	Sistemas com interfaces estáveis, equipes pequenas
Testing Trophy	Apps web/RN com data fetching + state management
Honeycomb	Microserviços, equipes distribuídas

Não há resposta universal. **Contexto define.**

Web testing vs Mobile testing

Aspecto	Web	Mobile
Plataformas	Chromium, Firefox, WebKit	iOS + Android (+ minoritárias)
Atualização	Refresh	Build + store
Fragmentação	Browsers + viewports	OS + devices + sensores
Sensores	Limitado	Câmera, GPS, biometria, NFC
Permissões	Cookies/storage	+ camera, location, contacts, push
Background	Limitado	Lifecycle complexo (foreground/background)

Tempo desta disciplina: **1 aula fundamentos + 2 aulas web + 1 aula mobile + 1 aula IA + 1 aula CI/CD e Projeto Final** (6 aulas no total).

Page Object Model (POM)

Padrão clássico. Encapsular interação com página em uma classe.

```
class LoginPage {
  emailInput = page.getByPlaceholder('Email');
  passwordInput = page.getByPlaceholder('Senha');
  submitButton = page.getByRole('button', { name: 'Entrar' });

  async login(email: string, password: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
  }
}

// Test
test('login flow', async () => {
  const loginPage = new LoginPage(page);
  await loginPage.login('user@email.com', 'secret');
  await expect(page).toHaveURL('/dashboard');
});
```

Screenplay Pattern

Evolução do POM. Modelo: **Actor, Ability, Task, Question.**

```
const user = Actor.named('Maria')
  .whoCan(BrowseTheWeb.using(page));

await user.attemptsTo(
  Login.withCredentials('user@email.com', 'secret'),
  Navigate.to('/dashboard')
);

await user.asks(TheTitle.ofPage()).should(equal('Dashboard'));
```

Mais verboso, mais expressivo. Bom para suítes grandes onde reuso é central.

Flakiness — definição

Teste flaky: mesmo código, mesma branch, comportamento diferente entre runs.

Causas mais comuns:

1. **Sincronização** (sleep em vez de wait)
2. **Race conditions** (animação, network)
3. **Test pollution** (ordem importa, leakage entre tests)
4. **Test environment instability** (CI lento, recursos compartilhados)
5. **Production bugs reais** (intermitência genuína)

Flakiness — métricas

```
Flake rate = runs flaky / total runs
```

Targets:

- Time elite: < 1% flake rate
- Time saudável: < 5%
- Time problemático: > 10%

```
# Detectar flaky em CI  
playwright test --repeat-each=3  
# Resultado: testes que falharam em 1 ou 2 (não 0 nem 3) são flaky
```

Flakiness — mitigação

- ✓ **Auto-waiting** (Playwright nativo, Maestro retry)
- ✓ **Idle Resources** (Espresso) ou `waitForExistence` (XCUITest)
- ✓ **Locators recomendados** (role, text, test-id) > seletores frágeis
- ✓ **Isolamento entre testes** (storageState, browser context)
- ✓ **Retry inteligente** em CI (1–2 max, com investigação)
- ✗ **Sleep fixo** (anti-pattern absoluto)
- ✗ **Retry em massa** (mascara bugs reais)
- ✗ **Skip silencioso** (esconde problemas)

Test Strategy Document

Documento que comunica estratégia de teste:

1. **Escopo** — o que testa, o que não testa
2. **Tipos de teste** — pirâmide aplicada
3. **Ferramentas** — stack escolhida com justificativa
4. **Ambientes** — dev, staging, prod, devices
5. **Métricas** — flake rate, escape rate, coverage
6. **Responsabilidades** — quem escreve, quem mantém
7. **CI/CD** — gates, sharding

| Documento vivo, revisado a cada release major.

Lab hands-on (90min)

Roteiro

1. Setup Node 22 + Playwright (10min)
2. Primeiro teste Playwright — `playwright codegen` (20min)
3. Refactor para Page Object Model + discussão: por que POM em vez de seletores soltos (20min)
4. Análise de pirâmide de uma suíte exemplo real + discussão em grupo (20min)
5. Documentar Test Strategy preliminar pro projeto final (20min)

Maestro (mobile) fica pra Aula 4 — hoje é só Playwright, senão vira setup de 2 ferramentas na 1ª aula.

Repo público: `github.com/jacksonsmith/puc-iec-qualidade-web-mobile` — o Lab Web+PWA (Aulas 2-3) usa `exercicios/01-lab-web-pwa-playwright/`

Leituras obrigatórias

- Crispin, L.; Gregory, J. — *Holistic Testing* (Pearson 2024), cap 1–3
- Dodds, K. — *Write Tests. Not Too Many. Mostly Integration.*
- ISO/IEC 25010:2023 — resumo guiado (link no Canvas)
- Spotify Engineering — *Testing of Microservices* (Honeycomb)

Bibliografia completa da disciplina: [BIBLIOGRAFIA.md](#)

Próxima aula — Aula 2 (01/10)

Playwright Avançado

- Arquitetura interna (CDP)
- Locators recomendados
- Network mocking + HAR replay
- storageState (auth reuse)
- Visual regression nativo
- Sharding 4-way em CI

Bons estudos!

✉ jackson.96@gmail.com (também canal Canvas)

🔗 linkedin.com/in/3jacksonsmith

🐙 github.com/jacksonsmith

Office hours: quintas 19h via Google Meet (link Canvas)